

# **CSA0613-Design and Analysis of Algorithms**

**Course Outcome: CO3\_AT1**

## **Experiments**

**Student Name: A. Bhavya(192424417)**

**Q3. Analyse the performance of Quick Sort under different partitioning schemes such as Lomuto and Hoare. Record execution time and number of swaps. Compare results and interpret differences. Justify which partitioning scheme provides better efficiency.**

### **Aim**

To analyze the performance of Quick Sort using Lomuto and Hoare partition schemes by measuring execution time and the number of swaps, comparing the results, and determining the more efficient partitioning technique.

### **Objective**

- Implement Quick Sort using Lomuto and Hoare partition schemes.
- Measure execution time for each scheme.
- Count the number of swaps performed.
- Compare the results.
- Analyze and justify the most efficient partitioning method.

### **Problem Statement**

Quick Sort is one of the most efficient divide-and-conquer sorting algorithms used for arranging data in ascending or descending order. The algorithm works by selecting a pivot element and partitioning the array into two subarrays such that elements smaller than the pivot are placed on one side and larger elements on the other side. The overall performance of Quick Sort largely depends on the partitioning technique used. Different partitioning schemes, such as Lomuto and Hoare partition methods, perform the partitioning process differently, resulting in variations in execution time and the number of swap operations. In this experiment, both Lomuto and Hoare partition schemes are implemented and tested on the same dataset. The execution time and total number of swaps are recorded and compared to analyze their efficiency. Based on the experimental results, the advantages and disadvantages of each

partitioning technique are interpreted, and the most efficient partition scheme is justified for practical applications and large datasets.

## **Algorithm**

### **Lomuto Partition**

1. Select the last element as the pivot.
2. Initialize index  $i = \text{low} - 1$ .
3. Traverse the array from low to high - 1.
4. If the current element is smaller than the pivot, increment  $i$  and swap.
5. Place the pivot in its correct position.
6. Recursively sort the left and right subarrays.

### **Hoare Partition**

1. Select the first element as the pivot.
2. Initialize two pointers:
  - $i = \text{low} - 1$
  - $j = \text{high} + 1$
3. Move  $i$  from left to right.
4. Move  $j$  from right to left.
5. Swap elements if  $i < j$ .
6. Repeat until pointers cross.
7. Recursively sort both partitions.

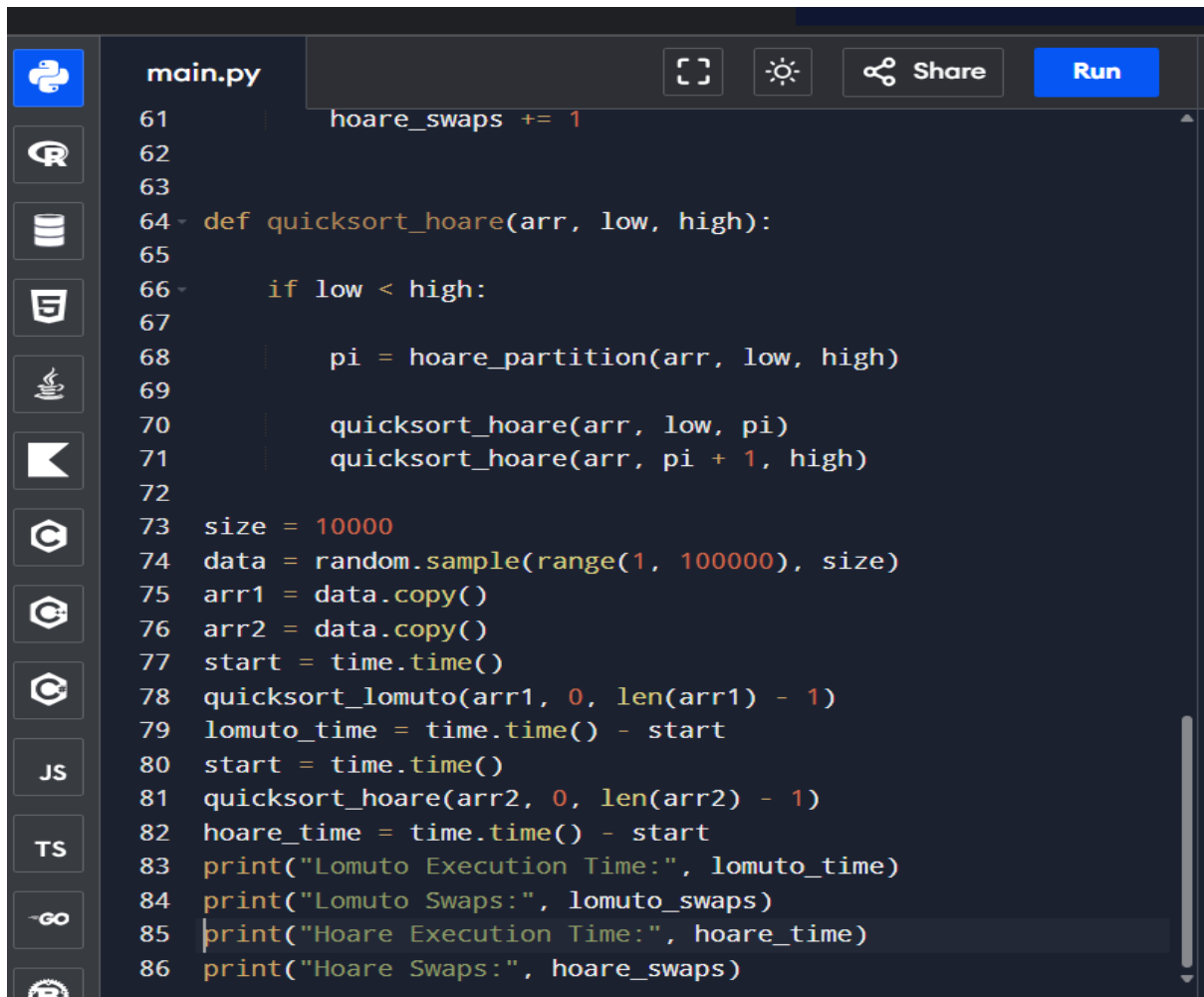
## **SOURCE CODE**

```
main.py  [ ] [ ] [ ] Share Run

1  import random
2  import time
3
4  lomuto_swaps = 0
5  hoare_swaps = 0
6
7  def lomuto_partition(arr, low, high):
8      global lomuto_swaps
9
10     pivot = arr[high]
11     i = low - 1
12
13     for j in range(low, high):
14
15         if arr[j] <= pivot:
16             i += 1
17
18             arr[i], arr[j] = arr[j], arr[i]
19             lomuto_swaps += 1
20
21     arr[i + 1], arr[high] = arr[high], arr[i + 1]
22     lomuto_swaps += 1
23
24     return i + 1
25
26
```

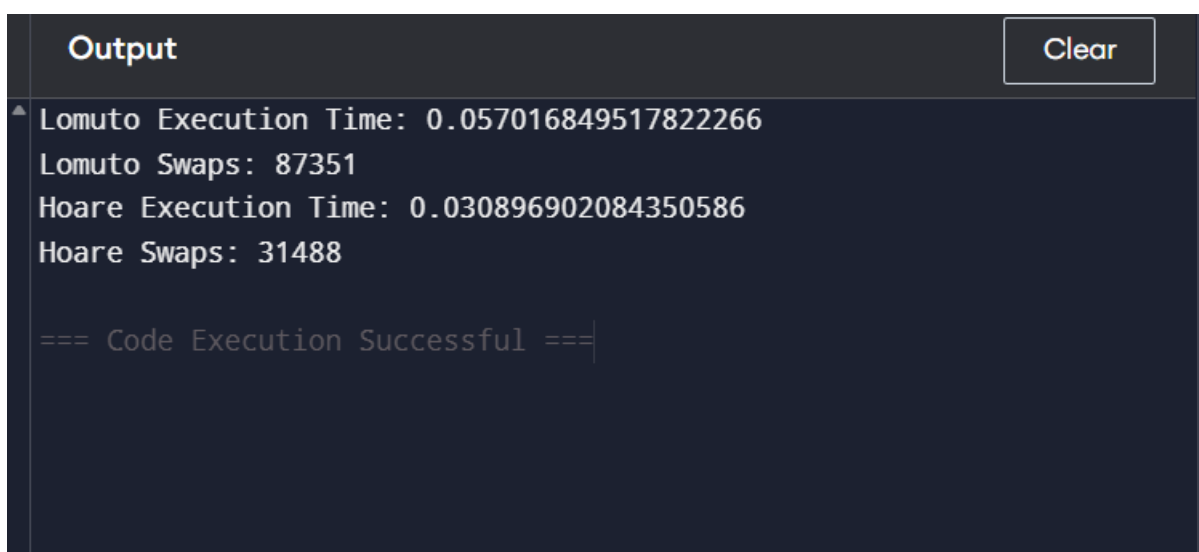
```
main.py  [ ] [ ] [ ] Share Run

--
27 def quicksort_lomuto(arr, low, high):
28
29     if low < high:
30
31         pi = lomuto_partition(arr, low, high)
32
33         quicksort_lomuto(arr, low, pi - 1)
34         quicksort_lomuto(arr, pi + 1, high)
35
36 def hoare_partition(arr, low, high):
37
38     global hoare_swaps
39
40     pivot = arr[low]
41
42     i = low - 1
43     j = high + 1
44
45     while True:
46
47         i += 1
48
49         while arr[i] < pivot:
50             i += 1
51
```



```
61     hoare_swaps += 1
62
63
64 def quicksort_hoare(arr, low, high):
65
66     if low < high:
67
68         pi = hoare_partition(arr, low, high)
69
70         quicksort_hoare(arr, low, pi)
71         quicksort_hoare(arr, pi + 1, high)
72
73 size = 10000
74 data = random.sample(range(1, 100000), size)
75 arr1 = data.copy()
76 arr2 = data.copy()
77 start = time.time()
78 quicksort_lomuto(arr1, 0, len(arr1) - 1)
79 lomuto_time = time.time() - start
80 start = time.time()
81 quicksort_hoare(arr2, 0, len(arr2) - 1)
82 hoare_time = time.time() - start
83 print("Lomuto Execution Time:", lomuto_time)
84 print("Lomuto Swaps:", lomuto_swaps)
85 print("Hoare Execution Time:", hoare_time)
86 print("Hoare Swaps:", hoare_swaps)
```

## OUTPUT:



```
Output Clear
^ Lomuto Execution Time: 0.057016849517822266
  Lomuto Swaps: 87351
  Hoare Execution Time: 0.030896902084350586
  Hoare Swaps: 31488

=== Code Execution Successful ===
```

## Analysis of Results

### Lomuto Partition

- Easy to understand and implement.
- Performs more swaps.
- Execution time is comparatively higher.
- Pivot selection can affect performance.

### Hoare Partition

- Performs fewer swaps.
- Faster execution.
- Better partition balance.
- More efficient for large datasets.

### Comparison

| Parameter       | Lomuto         | Hoare          |
|-----------------|----------------|----------------|
| Pivot Position  | Last Element   | First Element  |
| Number of Swaps | More           | Less           |
| Execution Time  | Higher         | Lower          |
| Performance     | Moderate       | Better         |
| Efficiency      | Less Efficient | More Efficient |

### Time Complexity

| Case         | Complexity    |
|--------------|---------------|
| Best Case    | $O(n \log n)$ |
| Average Case | $O(n \log n)$ |
| Worst Case   | $O(n^2)$      |

## Space Complexity

| Algorithm  | Complexity  |
|------------|-------------|
| Quick Sort | $O(\log n)$ |

## Interpretation

The experimental results indicate that the Hoare partition scheme performs better than the Lomuto partition scheme in terms of execution time and the number of swaps. Lomuto partition requires more swap operations, which increases the overall execution time. In contrast, Hoare partition minimizes unnecessary swaps by using two pointers that move from opposite ends of the array, resulting in faster execution. Although both schemes have the same average time complexity of  $O(n \log n)$ , Hoare partition demonstrates better practical performance, especially for large datasets.

## Conclusion

From the analysis, it can be concluded that the Hoare partition scheme is more efficient than the Lomuto partition scheme. Hoare partition performs fewer swaps and takes less execution time, making it more suitable for sorting large datasets. While Lomuto partition is simpler to implement, Hoare partition provides better optimization and overall performance. Therefore, Hoare partition is the preferred choice for Quick Sort implementations where efficiency is important.